Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

THE EVOLUTION OF MACHINE LEARNING AND AI

Week 3 Quiz Summary

How Machines Learn: everything the quiz can ask, in plain English

How to use this page. Week 3 is examined in the **Week 6 quiz** (50% of the module). The quiz is **multiple choice**. Everything it can ask about Week 3 is in the **eleven points** below, in the order of the lecture.

Read the boxes (about 20 minutes). Do the questions (about 20 minutes). Mark them. Re-read only the points you got wrong.

If you remember one thing. Machine learning is fitting a function to data by minimising a loss: guess, score the guess, share out the blame, take a small step downhill, repeat. Everything else this week is a choice about **which shapes** the answer may take, **how a mistake is scored**, and **how the search is done**.

1 Part 1. The eleven points

1.1 What learning actually means

A learning system is handed examples and finds a **rule** that turns inputs into outputs. It guesses a rule, scores how badly the guess does with **one number** (the **loss**), adjusts the guess slightly, and repeats. The loop has four stages: **forward** (guess), **loss** (score), **backward** (assign blame to every weight), **update** (step downhill). Every later week changes the first box and leaves the rest almost alone.

1.2 The three decisions

Every learning system is defined by three choices. The **hypothesis class**: which shapes of rule are allowed at all (typing a model's name is choosing it, and more data cannot add a shape the class does not contain). The **loss**: how a wrong answer becomes one number. The **optimiser**: how the space of rules is searched. **Parameters** are learned by the algorithm; **hyperparameters** are chosen by you, and a default is a choice made by nobody with your name on it. **Supervised** learning has the right answers attached to the examples; **unsupervised** learning looks for structure without them.

1.3 Lines: linear and logistic regression

Linear regression fits a straight line (or flat surface) and usually scores it by **squared error**, which is a decision: one large error counts for more than several small ones. Logistic regression squashes the same straight-line score through an S-shaped **sigmoid** to give a probability, and is scored by **cross-entropy**, which punishes **confident** wrong answers severely. Despite the curve it is still a **linear classifier**: its boundary is a straight line. Fit these first, always: readable, fast, and the baseline everything else must beat.

1.4 Boxes and margins: trees, forests and SVMs

A **decision tree** asks yes/no questions about one feature at a time, choosing each split to make the groups purer (**Gini impurity**). Left unlimited it memorises: **1.000** on training data, **0.857** on test. A **random forest** averages hundreds of deliberately different trees, made different by **bootstrap samples** of the rows and **random subsets of features** at each split. Averaging cuts variance: it also scores 1.000 on training and still generalises (0.890). A **support vector machine** finds the **widest empty corridor** between classes; only the **support vectors** on its edge matter. The **kernel trick** gets the benefit of lifting the data into a bigger space where a flat cut works, without ever computing that space. **C** sets how much a mistake costs. In the lab an RBF-kernel SVM beat the neural network and had the smallest train-test gap of all seven models.

1.5 The shape of the assumption

Same data, four boundaries: straight lines, right-angled boxes, a widest corridor, smooth bends. Each family is an **assumption about the shape of the truth**. This is Week 2′s **second sense of bias**, **inductive bias**, and it is necessary: a learner with no assumption cannot choose between the infinitely many rules that fit the examples. When the assumption matches reality you need less data; when it does not, more data will not help.

1.6 Capacity: underfitting and overfitting

Underfitting: high error on training **and** test data; the model is the wrong shape. **Overfitting**: low training error, high test error; it memorised the noise. **Report the gap**: one accuracy number is not a result, two numbers and their difference is. The famous case: a degree-18 polynomial had the **lowest training error** of three (0.012) and

a true error of **1499.8**. **Bias-variance**: error = bias² + variance + noise, meaning wrong shape, too sensitive to the particular sample, and the part nobody can remove. Perfect training accuracy is a symptom, not an achievement, unless you are averaging an ensemble.

1.7 The perceptron and its wall

One artificial neuron: weigh the inputs, add, threshold. Its learning rule nudges the weights after each mistake. The **convergence theorem** promises it will find a separating line **if one exists**: that condition is the whole story. **XOR** has four points no straight line separates, so the perceptron never gets more than **3 of 4** right, however long it trains: the failure is structural. One **hidden layer** re-represents the four points so that a line does work. A hidden layer buys a **learned change of coordinates**, not just more parameters. The escape route was obvious in 1969; **training** it took until 1986.

1.8 Gradient descent and the learning rate

Update: new weight = old weight minus **learning rate** × slope of the loss. It is purely **local**: it knows the slope where it stands and nothing else. **Batch**, **stochastic** and **mini-batch** use all, one, or a handful of examples per step; mini-batch won because it **fits the hardware**. For the bowl $f(x) = x^2/2$ the update is $x \leftarrow (1 - \eta)x$, so it converges only for $\mathbf{0 < \eta < 2}$. Four failure modes: **crawling** (too small), **converged**, **oscillating**, **diverging** (too big); crawling and oscillating can show **identical losses**. In high dimensions local minima are rare and **saddle points** are the real obstacle. **Momentum** is a decaying memory of past steps: about six times faster to the right neighbourhood, and it overshoots.

1.9 Backpropagation and the vanishing gradient

The **credit assignment problem**: which of thousands of weights caused the error? **Backpropagation** is the chain rule applied right to left, **reusing** partial results, which is what makes it efficient. It **computes a gradient**; it does not learn: **gradient descent does the learning**. The forward pass must **store** its intermediate values, which is why training needs far more memory than using a model. Check a hand-written gradient numerically, because a wrong derivative fails silently. Stacked layers with no activation collapse into one linear layer. Sigmoid's slope is at most **0.25**, so through ten layers the signal shrinks by about a million: the **vanishing gradient**. ReLU's slope is exactly 1 where active and 0 where **dead**: every fix creates the next problem.

1.10 PyTorch, and the trap that raises no error

A framework gives you **automatic differentiation** (it records every operation, a receipt, and reads it backwards applying the chain rule), hardware (the same code on a GPU) and tested components. The training loop is five lines: **zero_grad**, forward, loss, **backward**, **step**. The trap: PyTorch **adds** new gradients to the old ones. Leave out **zero_grad** and every step uses the running sum of every gradient so far. Nothing crashes and nothing warns: the lab's two runs ended at 0.244 and 0.250, and only one of them learned. loss.backward() replaced eight lines of chain rule, not the idea behind them.

1.11 Why it waited until 2012

Backpropagation worked in 1986, yet deep learning arrived in 2012. The algorithm was not the missing piece: the **data** (ImageNet, 2009) and the **compute** (graphics cards made programmable, 2007) were. AlexNet was 1986′s algorithm on 2009′s data and 2007′s silicon. This is Week 1′s three supplies seen from inside the machine.

2 Part 2. The seven questions for any model you train

You may be asked to apply these, so read each one as a test you could run.

#	Question
1	What shape of answer can this model represent, and does the truth have that shape?
2	What is the loss, and what does it say I care about?
3	What is my train-test gap, and did I look at it before celebrating?
4	Is my learning rate in the band between crawling and diverging?
5	Did I check the gradient once, numerically?
6	Would a simpler model have done?
7	Which hyperparameters did I choose, and can I defend each in a sentence?

3 Part 3. Twenty practice questions

Same format as the quiz: one right answer each. Answers on the last page.

1. A team's straight-line classifier is stuck at 70% accuracy on data whose true boundary is a circle. A manager proposes collecting ten times more data. What is the best response?

A. More data will help, because accuracy always improves with data
B. Use a bigger learning rate so training finishes sooner
C. Train for more epochs, because the model has not converged
D. The hypothesis class cannot draw a circle, so more data will not fix it; change the shape of model

2. Which of these is a hyperparameter rather than a parameter?

A. The learning rate you set before training
B. A weight inside a neural network
C. The slope of a fitted regression line
D. A split threshold the tree learned from the data

3. Logistic regression produces an S-shaped curve. Why is it still called a linear classifier?

A. Because it is trained by gradient descent
B. Because its decision boundary is a straight line; the S-curve only turns the score into a probability
C. Because it uses squared error
D. Because it can only handle one feature

4. Compared with squared error, what does cross-entropy do to a model that is confidently wrong?

A. Ignores it, because confidence is not measured
B. Punishes it less, to keep training stable
C. Punishes it severely, so confident mistakes dominate the loss
D. Treats it the same as an unsure mistake

5. An unlimited decision tree scores 1.000 on training data and 0.857 on test data. What is happening?

A. It has memorised the training data, noise included, and generalises worse
B. It has underfitted, because 1.000 is suspiciously round
C. The test set must contain errors
D. Nothing: 1.000 on training proves the model is right

6. A random forest also scores 1.000 on training data, yet generalises well (0.890 on test). How is that possible?

A. Forests never overfit, whatever the data
B. The forest is tested on easier data
C. It uses squared error instead of Gini impurity
D. Each tree memorises differently, because of bootstrap rows and random feature subsets, and averaging cancels their mistakes

7. In a support vector machine, you delete every training point except the support vectors and retrain. What happens?

A. The boundary moves towards the deleted points
B. The boundary is unchanged, because only the points on the edge of the corridor decide it
C. Training fails, because the model needs all the data
D. The margin becomes twice as wide

8. What does the kernel trick let an SVM do?

A. Use a flat cut in a much bigger space without ever computing the points in that space
B. Train without any labels
C. Replace the margin with a probability
D. Turn a classifier into a regression model

9. Week 3 calls a model family's built-in shape an inductive bias. Which statement matches the week?

A. Inductive bias is a fairness problem and should be removed
B. Inductive bias only matters for neural networks
C. Every learner needs one; when it matches the truth you need less data, and when it does not, more data will not help
D. Inductive bias is the same as statistical bias in an estimate

10. Three polynomials are fitted. The degree-18 one has the lowest training error and a true error over a thousand times larger than the others. Which would you deploy?

A. The degree-18 model, because it fits the data best
B. Whichever trained fastest
C. Any of them, because training error is what counts
D. Not the degree-18 model: look at the gap between training and true error, and pick the model with the best test result

11. Which sequence is the training loop Week 3 says sits under every model in the module?

A. Forward (guess), loss (score), backward (blame every weight), update (step downhill), repeat
B. Collect data, label it, deploy it, retrain it
C. Loss, forward, update, backward, repeat
D. Backward, forward, loss, update, repeat

12. Why can a single perceptron never get all four XOR points right, however long it trains?

A. Its learning rate is too small
B. No straight line separates the four XOR points, and one perceptron can only draw a straight line
C. XOR has too few examples to learn from
D. The convergence theorem says perceptrons never converge

13. What does adding a hidden layer actually buy, in Week 3′s words?

A. More memory for the training data
B. A faster learning rate
C. A learned change of coordinates, so that a straight line works in the new representation
D. Protection against overfitting

14. Gradient descent on $f(x) = x^2/2$ uses the update $x \leftarrow (1 - \eta)x$. For which learning rates does it converge?

A. Any positive learning rate
B. Only learning rates above 2
C. Only a learning rate of exactly 1
D. Learning rates between 0 and 2

15. Two training runs finish with identical losses. One learning rate was far too small, the other too large. How can that be?

A. Crawling and oscillating can leave the loss at the same value for opposite reasons
B. The loss function must be wrong
C. Identical losses mean identical models
D. It cannot happen, so the runs were mislabelled

16. Backpropagation worked in 1986, yet deep learning took off in 2012. In Week 3′s account, what was missing?

A. A correct version of the chain rule
B. A way to train hidden layers
C. The data and the compute: AlexNet was 1986′s algorithm on 2009′s data and 2007′s silicon
D. The idea of stacking more than one layer

17. Which statement about backpropagation is correct?

A. It is the step that updates the weights
B. It computes the gradient efficiently by applying the chain rule right to left; gradient descent does the learning
C. It replaces the need for a loss function
D. It only works for one-layer networks

18. Why does training a network need far more memory than using it?

A. The forward pass must store intermediate values so the backward pass can use them
B. Training uses a larger model
C. Weights are copied for every example
D. The loss is stored for every epoch

19. A ten-layer network with sigmoid activations barely learns in its early layers. What is the most likely cause?

A. The learning rate is too large
B. Too little training data
C. The early layers have too many weights
D. The vanishing gradient: sigmoid's slope is at most 0.25, so the signal shrinks layer after layer

20. A student's PyTorch loop runs four hundred epochs with no error, but the model barely learns. The loop has loss.backward() and optimiser.step() but no optimiser.zero_grad(). What is wrong?

A. Nothing; zero_grad() is optional decoration
B. The model needs more epochs
C. Gradients accumulate, so every step uses the running sum of all past gradients
D. The learning rate must be lowered

4 Answers

#	Answer	Why
1	**D**	The hypothesis class decides which shapes are possible at all; more data cannot add a missing shape. Points 2 and 5.
2	**A**	Parameters are learned by the algorithm; hyperparameters are chosen by you. Point 2.
3	**B**	The sigmoid squashes a straight-line score; the boundary stays straight. Point 3.
4	**C**	Cross-entropy is a decision to punish confident wrongness hard. Point 3.
5	**A**	Low training error with a much higher test error is the signature of overfitting. Points 4 and 6.
6	**D**	Averaging many low-bias, high-variance models with uncorrelated errors drives the variance down. Point 4.
7	**B**	Only the support vectors matter; the rest can be discarded without changing the solution. Point 4.
8	**A**	It needs only comparisons between pairs of points, which a kernel computes directly. Point 4.
9	**C**	Inductive bias is Week 2′s second sense of bias: necessary, and a bet about the world. Point 5.
10	**D**	Report the gap; the lowest training error was the worst model on the page. Point 6.
11	**A**	Guess, score, blame, step; later weeks change the first box and leave the rest almost alone. Point 1.
12	**B**	The failure is structural: at most 3 of 4, because no line separates XOR. Point 7.
13	**C**	A hidden layer re-represents the points; it is not just more parameters. Point 7.
14	**D**	Each step multiplies x by $(1 - \eta)$, which shrinks only when $0 < \eta < 2$. Point 8.
15	**A**	Two of the four failure modes can look the same in the loss; sweep the learning rate. Point 8.
16	**C**	The algorithm was ready; the supplies were not. Point 11.
17	**B**	Backpropagation computes a gradient; it does not learn. Point 9.
18	**A**	Backpropagation needs the stored intermediates; inference can throw them away. Point 9.
19	**D**	Multiplying ten slopes of at most 0.25 shrinks the gradient by about a million. Point 9.
20	**C**	PyTorch adds gradients rather than replacing them; the trap raises no error. Point 10.

Scored under 15? Re-read the points for the questions you missed, then try again tomorrow. The answers say which point each question came from.